

Q36. What is the difference between JDK, JRE, and JVM?

Simple Answer

JDK, JRE, and JVM are the three main components of the Java platform.

- **JDK (Java Development Kit)** is used to develop Java applications.
- **JRE (Java Runtime Environment)** provides the environment to run Java applications.
- **JVM (Java Virtual Machine)** executes Java bytecode.

Detailed Explanation

JDK

JDK contains everything needed to develop Java applications:

- JRE
- Compiler (`javac`)
- Debugger
- Development tools

JRE

JRE contains:

- JVM
- Core Java Libraries

It is used only to run Java applications.

JVM

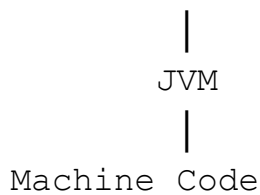
JVM converts bytecode into machine code and executes it.

It also manages:

- Memory
- Garbage Collection
- Class Loading

Architecture

```
Java Source (.java)
    |
    javac
    |
Bytecode (.class)
```



Real-Time Example

Suppose you develop a Spring Boot application.

- You use **JDK** to write and compile the code.
- The generated `.class` files are deployed.
- The server uses **JRE**, and the **JVM** executes the bytecode.

Code Example

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

Interview Tip

A classic interview question:

Does JDK contain JRE?

Yes.

Does JRE contain JVM?

Yes.

Key Points

✓ JDK = Development

✓ JRE = Execution Environment

✓ JVM = Executes Bytecode

✓ JDK $\supset$ JRE $\supset$ JVM

Q37. What is JVM? How does it work?

Simple Answer

The **Java Virtual Machine (JVM)** is a virtual machine that executes Java bytecode.

It enables Java's "**Write Once, Run Anywhere**" feature by converting platform-independent bytecode into platform-specific machine code.

Detailed Explanation

The JVM performs several important tasks:

- Loads classes using the Class Loader.
- Verifies bytecode for security.
- Allocates memory.
- Executes bytecode.
- Performs Garbage Collection.

JVM Architecture

```
Class Loader
  |
Bytecode Verifier
  |
Execution Engine
  |
Garbage Collector
```

Real-Time Example

A banking application developed on Windows can be deployed on Linux without recompiling.

Only the JVM changes based on the operating system.

Code Example

```
System.out.println("Running inside JVM");
```

Interview Tip

Remember:

JVM is platform-dependent.

Bytecode is platform-independent.

Key Points

- ✓ Executes bytecode.
- ✓ Performs Garbage Collection.
- ✓ Loads classes.
- ✓ Enables platform independence.

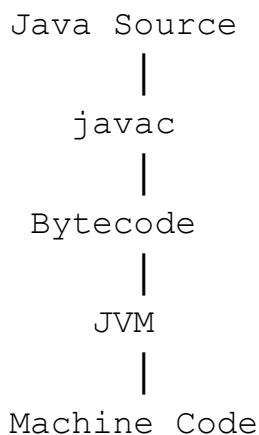
Q38. Why is Java Platform Independent?

Simple Answer

Java is platform independent because it compiles source code into **bytecode**, which can run on any operating system that has a compatible JVM.

Detailed Explanation

Compilation process:



Unlike C or C++, Java does not compile directly into machine code.

Instead, the JVM converts bytecode into machine code specific to the operating system.

Real-Time Example

A Java application compiled on Windows can run on:

- Linux
- macOS
- Unix

without recompiling.

Code Example

```
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("Platform Independent");  
    }  
}
```

Interview Tip

A common interview question:

Who makes Java platform independent?

Answer: JVM

Key Points

- ✓ Bytecode is platform-independent.
- ✓ JVM is platform-specific.
- ✓ Same `.class` file runs on multiple operating systems.

Q39. What is Bytecode?

Simple Answer

Bytecode is the intermediate code generated by the Java compiler (`javac`) after compiling a `.java` file.

The JVM executes bytecode.

Detailed Explanation

Compilation flow:

`Main.java`

↓

`javac`

↓

`Main.class`

↓

JVM

Bytecode is not machine code.

It is a platform-independent instruction set understood by the JVM.

Real-Time Example

When a developer builds a Java application, the generated `.class` files are deployed to production servers.

These `.class` files contain bytecode.

Code Example

```
javac Main.java
```

Produces:

```
Main.class
```

Interview Tip

Remember:

Java source code → Bytecode → Machine code

Key Points

- ✓ Generated by `javac`.
- ✓ Stored in `.class` files.
- ✓ Executed by the JVM.
- ✓ Platform-independent.

Q40. What is the difference between Stack Memory and Heap Memory?

Simple Answer

Java uses two major memory areas:

- **Stack Memory** stores method calls and local variables.
- **Heap Memory** stores objects and instance variables.

Detailed Explanation

Stack	Heap
Stores local variables	Stores objects
Thread-specific	Shared by all threads
Automatically managed	Managed by Garbage Collector
Faster access	Larger memory area

Real-Time Example

```
Employee emp = new Employee();
```

- emp (reference variable) is stored in the **Stack**.
- The Employee object is created in the **Heap**.

Code Example

```
class Employee {  
  
    String name = "Murali";  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Employee emp = new Employee();  
    }  
}
```

Interview Tip

This is one of the **Top 10 Core Java interview questions**.

A simple way to remember:

- **Stack = Methods + Local Variables**
- **Heap = Objects + Instance Variables**

Key Points

- ✓ Stack stores local variables.
- ✓ Heap stores objects.
- ✓ Heap is managed by the Garbage Collector.
- ✓ Every object is created in the Heap.

Q41. What are the different Memory Areas in JVM?

Simple Answer

The JVM divides memory into different areas to efficiently manage program execution.

The major memory areas are:

- Heap
- Stack
- Method Area (Metaspace)
- Program Counter (PC) Register
- Native Method Stack

Detailed Explanation

1. Heap Memory

- Stores objects and instance variables.
- Shared among all threads.
- Managed by the Garbage Collector.

2. Stack Memory

Stores:

- Local variables
- Method calls
- References

Each thread has its own stack.

3. Method Area (Metaspace)

Stores:

- Class metadata
- Static variables
- Method information
- Constant pool

Since Java 8, the Method Area is implemented as **Metaspace**.

4. Program Counter Register

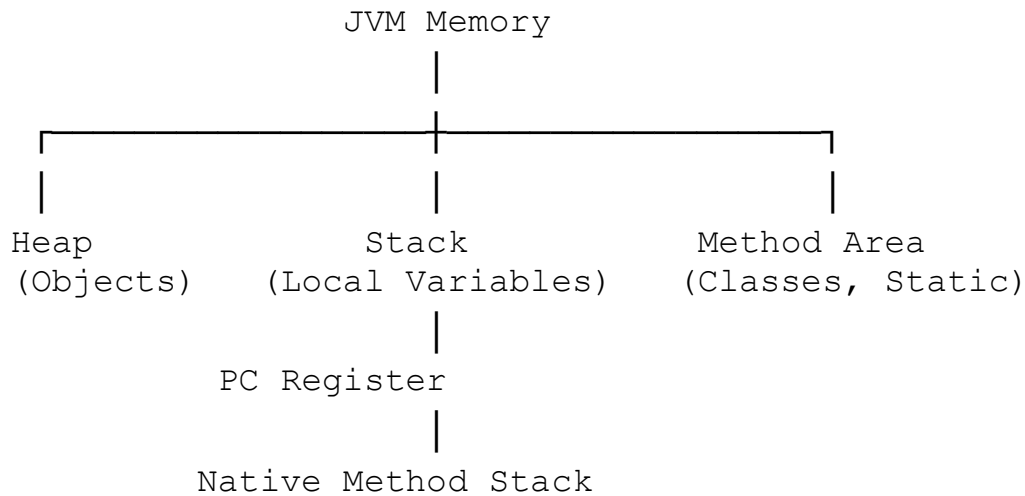
Stores the address of the current instruction being executed by the thread.

Each thread has its own PC Register.

5. Native Method Stack

Used when Java code calls native methods written in languages like C or C++ through JNI (Java Native Interface).

JVM Memory Diagram



Real-Time Example

When an employee logs into an application:

- Employee object → Heap
- Local variables → Stack
- Employee class → Method Area
- Static company name → Method Area (Metaspace)

Code Example

```
class Employee {  
    static String company = "CTS";  
    String name = "Murali";  
    void display() {  
        int age = 30;  
        System.out.println(name);  
    }  
}
```

Memory allocation:

```
Heap
-----
Employee Object

Stack
-----
display()
age
emp reference

Method Area
-----
Employee Class
company
```

Interview Tip

One of the favorite interview questions:

Where are static variables stored?

Answer (Java 8+):

In the **Method Area (Metaspace)**.

Key Points

- ✓ Heap stores objects.
- ✓ Stack stores local variables.
- ✓ Metaspace stores class metadata.
- ✓ Every thread has its own Stack and PC Register.

Q42. What is Garbage Collection in Java?

Simple Answer

Garbage Collection (GC) is the automatic process of removing unused objects from Heap memory.

It helps prevent memory leaks and automatically manages memory without requiring programmers to free memory manually.

Detailed Explanation

Whenever an object is no longer referenced by any part of the program, it becomes **eligible for garbage collection**.

The Garbage Collector periodically identifies these unreachable objects and reclaims their memory.

Advantages:

- Automatic memory management
- Prevents memory leaks
- Reduces manual coding errors

Real-Time Example

Suppose a user logs into an application.

An `Employee` object is created.

After logout, if no reference points to that object, it becomes eligible for garbage collection.

Code Example

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Employee emp = new Employee();  
  
        emp = null;  
  
        System.gc();    // Request JVM to run GC  
    }  
}
```

Interview Tip

Interviewers often ask:

Does `System.gc()` guarantee Garbage Collection?

Answer:

No.

It is only a request to the JVM.

The JVM decides whether to perform garbage collection.

Key Points

- ✓ Removes unused objects.
- ✓ Works only on Heap memory.
- ✓ Automatic memory management.
- ✓ `System.gc()` is only a request.

Q43. What is the difference between `==` and `equals()`?

Simple Answer

`==` compares **memory references** (or primitive values), whereas `equals()` compares the **contents** of objects.

Detailed Explanation

`==`

- Primitive types → compares actual values.
- Objects → compares memory addresses.

`equals()`

- Compares object contents.
- Can be overridden for custom comparison.

Real-Time Example

Two employees may have the same Employee ID and Name.

Although they are different objects in memory, logically they represent the same employee.

The `equals()` method can be overridden to compare their data instead of their memory addresses.

Code Example

```
String s1 = new String("Java");  
String s2 = new String("Java");  
  
System.out.println(s1 == s2);  
  
System.out.println(s1.equals(s2));
```

Output

```
false  
true
```

Interview Tip

Very common interview question:

What does `==` compare for objects?

Answer:

Memory references.

Key Points

- ✓ `==` compares references.
- ✓ `equals()` compares contents.
- ✓ `equals()` can be overridden.

Q44. Why should we override both `equals()` and `hashCode()`?

Simple Answer

Whenever two objects are considered equal using `equals()`, they must return the same `hashCode()`.

Otherwise, collections such as `HashMap` and `HashSet` may not work correctly.

Detailed Explanation

Hash-based collections use:

1. `hashCode()` to find the bucket.
2. `equals()` to compare objects inside the bucket.

If only `equals()` is overridden, equal objects may end up in different buckets.

Real-Time Example

Suppose Employee ID is the unique identifier.

Two Employee objects with the same ID should be treated as equal.

Therefore:

- Override `equals()`
- Override `hashCode()`

Code Example

```
@Override
public boolean equals(Object obj) {

    // comparison logic
}

@Override
public int hashCode() {

    return id;
}
```

Interview Tip

One of the most frequently asked HashMap interview questions.

Remember:

`equals()` decides equality.

`hashCode()` decides bucket location.

Key Points

✓ Used by HashMap and HashSet.

✓ Equal objects must have equal hash codes.

✓ Improves collection performance.

Q45. What is the difference between **final**, **finally**, and **finalize()** ?

Simple Answer

Although their names are similar, they serve completely different purposes.

Keyword/Method	Purpose
<code>final</code>	Prevents modification
<code>finally</code>	Executes after <code>try-catch</code>
<code>finalize()</code>	Invoked by the Garbage Collector before object destruction (deprecated)

Detailed Explanation

final

Used with:

- Variables
- Methods
- Classes

finally

Always executes after the `try-catch` block, whether an exception occurs or not.

Commonly used for resource cleanup.

finalize()

A method that was called by the Garbage Collector before reclaiming an object.

Note: `finalize()` is **deprecated** and should not be used in modern Java applications. Prefer `try-with-resources` or explicit cleanup.

Real-Time Example

Imagine reading a file.

- `try` → Read the file.
- `catch` → Handle exceptions.
- `finally` → Close the file.

Code Example

```
try {  
  
    int result = 10 / 0;  
  
} catch (Exception e) {  
  
    System.out.println("Exception");  
  
} finally {  
  
    System.out.println("Finally Executed");  
}
```

Output

```
Exception  
Finally Executed
```

Interview Tip

Interviewers often ask:

Is `finally` always executed?

Answer:

Almost always. It may not execute in rare situations such as a JVM crash or a call to `System.exit()` before reaching the `finally` block.

Key Points

- ✓ `final` → Restricts modification.
- ✓ `finally` → Executes after `try-catch`.

✓ `finalize()` → Deprecated; avoid using it.